


**END SEMESTER ASSESSMENT (ESA)**
**B.TECH. (CSE)**
**III SEMESTER**

**UE25CS251A**
**DIGITAL LOGIC AND COMPUTER DESIGN LABORATORY**

**PROJECT REPORT**

**ON**

# "PIPELINED MIPS PROCESSOR WITH HAZARD DETECTION"

SUBMITTED BY


| NAME | SRN |
|---|---|
| 1) Jayanth K P | PES2UG25CS231 |
| 2) Jeevan N | PES2UG25CS232 |
| 3) Kadiyala Sai Arjun | PES2UG25CS244 |
| 4) Karthick Manju | PES2UG25CS250 |


**AUGUST – DECEMBER 2026**


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

**ELECTRONIC CITY CAMPUS,**

**BENGALURU – 560100, KARNATAKA, INDIA**

| TABLE OF CONTENTS | | |
|---|---|---|
| Sl.No | TOPIC | PAGE No |
| 1. | PROBLEM STATEMENT | 3 |
| 2. | CIRCUIT DIAGRAM | 4 |
| 3. | FSM<br>(only if Applicable to assigned project) | 6 |
| 4. | LITERATURE SURVEY<br>(Brief Description of the concept behind the project executed) | 7 |
| 5. | MAIN VERILOG CODE | 9 |
| 6. | TEST BENCH FILE | 16 |
| 7. | SCREEN SHOTS OF THE OUTPUT | 19 |
| 8. | REFERENCES<br>(Links to sites, papers or other referred material) | 21 |

# PROBLEM STATEMENT

The aim of this project is to design, implement and verify, in Verilog HDL, a **five-stage pipelined MIPS processor** (Instruction Fetch, Instruction Decode, Execute, Memory Access and Write Back) that **detects and resolves all three classes of pipeline hazards**, and to demonstrate its behaviour through simulation in QuestaSim.

Pipelining overlaps the execution of several instructions to increase throughput. The overlap, however, creates *hazards*: situations in which the next instruction cannot safely execute in its assigned clock cycle. Without dedicated hardware the processor would read stale operands or fetch wrong-path instructions and produce incorrect results. This project builds that hardware:

- **Data hazards.** A read-after-write dependency between nearby instructions is resolved by a **forwarding unit**, which routes results held in the EX/MEM and MEM/WB registers straight back to the ALU inputs, together with a write-back-to-decode bypass for a value that is written in WB in the same cycle it is read in ID. A **load-use hazard** (an instruction that needs a register loaded by the instruction immediately before it) cannot be forwarded in time, so the **hazard detection unit** stalls the pipeline for one cycle and inserts a bubble.
- **Control hazards.** The conditional branches BEQ and BNE are resolved in the Execute stage. When a branch is taken the PC is loaded with the branch target and the two wrong-path instructions already fetched are flushed from the IF/ID and ID/EX registers (a two-cycle penalty). Not-taken branches cost nothing.
- **Structural hazards.** These occur when two instructions need the same hardware in the same cycle. They are avoided by using separate instruction and data memories and a register file that supports two reads and one write per cycle. The design also flags the cycles in which such a conflict would have occurred, so that it can be observed in simulation.

**Supported instructions:** add, sub, and, or, slt, addi, lw, sw, beq and bne.

## Objectives

- Implement the five pipeline stages and the four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB).
- Implement the hazard detection unit (load-use stall and branch flush) and the forwarding unit.
- Provide hazard indicator signals for data, control and structural hazards so each can be seen in the waveform.
- Verify the design with a self-checking testbench whose program is built to trigger every hazard type, and observe the stalls, flushes and forwarding in QuestaSim.

**Tools used:** Verilog HDL; QuestaSim (compilation, simulation and waveform analysis).

# CIRCUIT DIAGRAM

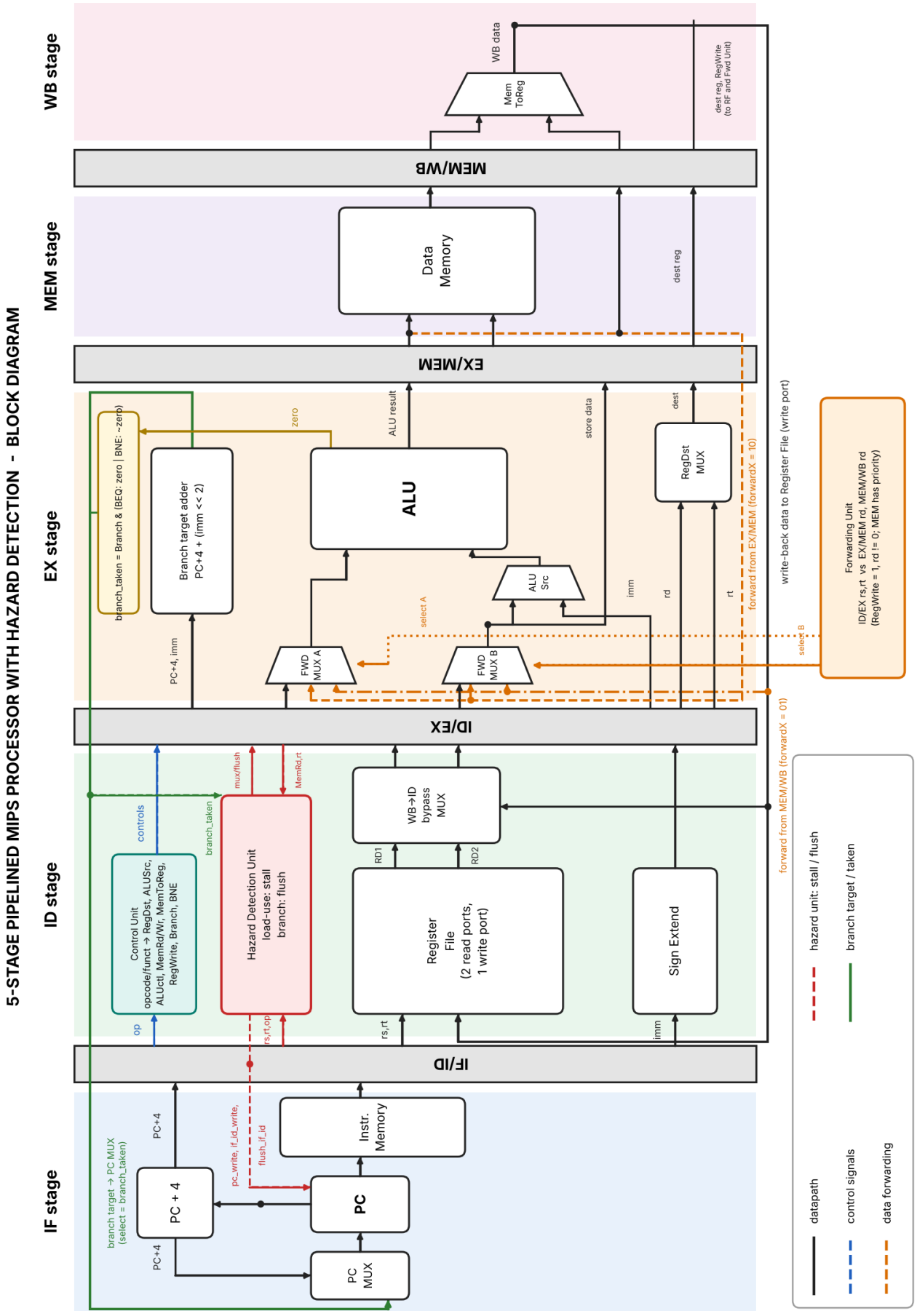



*Figure 1: Block diagram of the pipelined processor (turn the page 90° clockwise to read).*

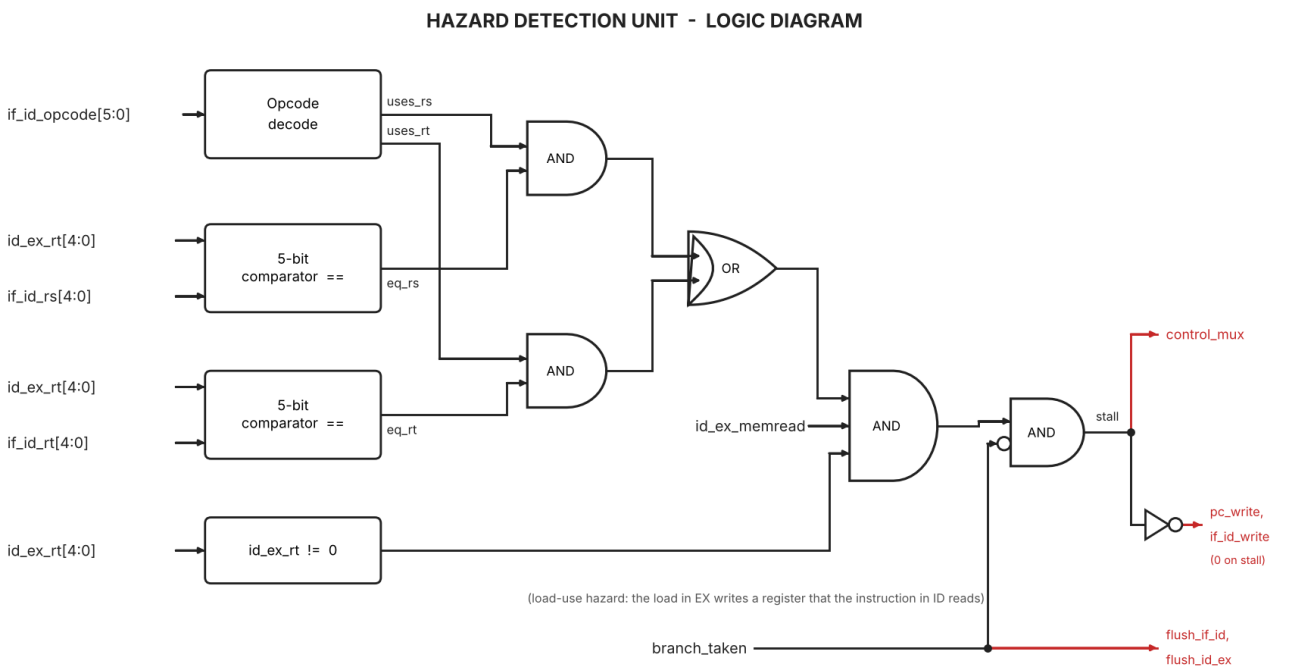



*Figure 2: Logic diagram of the hazard detection unit. A load-use hazard exists when the instruction in EX is a load (id_ex_memread) whose destination id_ex_rt (not $0) is read by the instruction in ID. The stall is suppressed while a branch is being taken, because the flush has priority.*

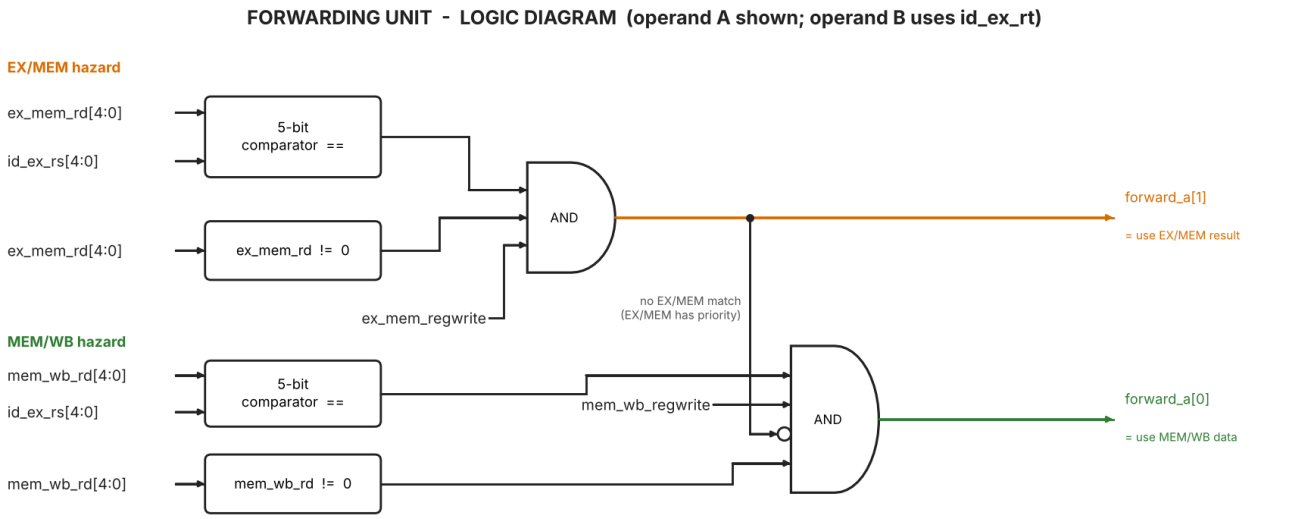



*Figure 3: Logic diagram of the forwarding unit for ALU operand A. EX/MEM has priority over MEM/WB, and register $0 is never forwarded.*

# FSM

**Not applicable to this project.**

The design does not use a finite state machine. Every instruction advances one pipeline stage per clock cycle, and all of the control is combinational: the control unit decodes the instruction in the ID stage, and the hazard detection unit and forwarding unit are purely combinational logic driven by the current contents of the pipeline registers (see the logic diagrams in Section 2). A stall or flush is therefore produced directly from the instructions in flight rather than from an explicit state.

# LITERATURE SURVEY

Pipelining is the fundamental technique by which modern processors raise instruction throughput. By overlapping the execution of successive instructions across the stages of fetch, decode, execute, memory access and write back, an ideal pipeline completes one instruction every clock cycle [1], [2]. The five-stage MIPS pipeline described by Patterson and Hennessy has become the standard reference organisation for teaching and for hardware design exercises, and it forms the basis of the present work [1], [4].

The principal obstacle to achieving this ideal throughput is the occurrence of *hazards*, which are conditions that prevent the next instruction from executing in its designated clock cycle. The literature classifies them into three categories: data, control and structural hazards [2]. The treatment of each category in the literature is summarised below.

**Data hazards.** A data hazard arises when an instruction depends on the result of a preceding instruction that has not yet been written back, giving rise to a read-after-write dependency. Two techniques are established for dealing with it. The first is *forwarding* (also called bypassing), in which a result is routed directly from the EX/MEM or MEM/WB pipeline register to the ALU input, which removes the majority of stalls without requiring the instruction to wait for the write-back stage [1], [2]. The second is *interlocking*, in which a hazard detection unit stalls the pipeline. Interlocking cannot be avoided for a load-use dependency, since the loaded value is not available until the end of the memory stage; a one-cycle stall must therefore be inserted even when forwarding is present [1]. Compiler-based instruction scheduling can reduce the number of such stalls, but a hardware interlock remains necessary to guarantee correctness [2]. Both mechanisms are implemented in this project.

**Control hazards.** A control hazard arises because the outcome of a branch is not known when the following instructions are fetched. The cost depends on the stage in which the branch is resolved: resolving it in the Execute stage, as in this project, incurs a penalty of two cycles when the branch is taken, whereas resolving it earlier in the Decode stage reduces the penalty at the expense of an additional comparator and extra forwarding paths [1]. Branch prediction reduces the penalty further. Static schemes (for example, predicting not taken) were extended by dynamic predictors that use the past behaviour of each branch, beginning with the saturating-counter schemes studied by Smith [6] and followed by correlating and more elaborate predictors described in [2]. The present design adopts the simplest correct scheme, namely predict-not-taken with a flush of the wrongly fetched instructions when a branch is taken.

**Structural hazards.** A structural hazard arises when two instructions require the same hardware resource in the same clock cycle, for example a single memory used for both instruction fetch and data access, or a register file with insufficient ports. The accepted solution is to replicate or to provide sufficient ports on the contended resource: separate instruction and data memories, and a register file with two read ports and one write port [1], [4]. The present design follows this approach, so such conflicts are avoided by construction, and indicator signals are provided to observe the cycles in which they would otherwise occur.

**Hardware description and verification.** Implementations of the pipelined MIPS processor in Verilog HDL, including the hazard detection and forwarding units, are presented in [3], [4] and [5]. Verification practice for designs of this size relies on self-checking testbenches containing directed test programs that deliberately exercise each hazard class, supported by inspection of simulation waveforms in a simulator such as QuestaSim [7]. This project follows that practice: the testbench executes a program constructed to trigger every hazard type, automatically checks the final register and memory contents, and records the number of stalls, flushes and forwarding events.

**Summary.** The reviewed literature establishes forwarding with a load-use interlock for data hazards, early resolution or prediction with flushing for control hazards, and resource duplication for structural hazards. The present project combines these established techniques in a single five-stage Verilog implementation and demonstrates each of them in simulation.

# MAIN VERILOG CODE:

The processor is built from the modules listed below. The remaining modules (register file, instruction and data memory, and the IF/ID, ID/EX, EX/MEM and MEM/WB pipeline registers) are part of the same project.

**mips_pipeline.v** - Top-level module: connects the five stages, pipeline registers and the hazard logic

```
// Top-level 5-stage pipelined MIPS: IF / ID / EX / MEM / WB
// Wires together the repo's modules.
//
// HAZARDS DETECTED AND HANDLED
//  DATA       forwarding_unit        EX/MEM->EX and MEM/WB->EX forwarding
//             hazard_detection_unit  load-use: stall PC + IF/ID, bubble in ID/EX
//             WB->ID bypass (here)   register file writes on the clock edge, so a
//                                    value in WB is not visible to ID that cycle
//  CONTROL    BEQ/BNE resolved in EX. branch_taken -> PC loads branch target and
//             hazard_detection_unit flushes IF/ID and ID/EX (2-cycle penalty)
//  STRUCTURAL avoided by design (separate instruction + data memory, register
//             file read/write in the same cycle). The struct_* wires below DETECT
//             when the conflict would have occurred so it can be shown/counted.
module mips_pipeline (
    input wire clk,
    input wire reset
);
    // ------------------------------------------------------------
    // Cross-stage signals, all declared up front (strict tools such as
    // QuestaSim reject use-before-declaration).
    // ------------------------------------------------------------
    wire        pc_write_hdu, if_id_write, control_mux_hdu;
    wire        flush_if_id, flush_id_ex;
    wire        branch_taken_ex;
    wire [31:0] branch_target_ex;

    wire [31:0] wb_data, wb_read_data, wb_alu_result;
    wire [4:0]  wb_write_reg;
    wire        wb_mem_to_reg, wb_reg_write;

    wire [31:0] ex_pc4, ex_rd1, ex_rd2, ex_imm;
    wire [4:0]  ex_rs, ex_rt, ex_rd;
    wire        ex_reg_dst, ex_alu_src, ex_mem_read, ex_mem_write;
    wire        ex_mem_to_reg, ex_reg_write, ex_branch;
    wire [2:0]  ex_alu_control;

    wire [31:0] mem_alu_result, mem_write_data, mem_read_data;
    wire [4:0]  mem_write_reg;
    wire        mem_reg_write, mem_mem_read, mem_mem_write, mem_mem_to_reg;

    // ------------------------------------------------------------------ IF
    wire [31:0] pc, pc_plus4_if, instr_if;
    assign pc_plus4_if = pc + 32'd4;

    program_counter pc_reg (
        .clk(clk), .reset(reset),
        .pc_write(pc_write_hdu),
        .branch_taken(branch_taken_ex),
        .branch_target(branch_target_ex),
        .next_pc(pc_plus4_if),
        .pc(pc)
    );

    instruction_memory imem (.address(pc), .instruction(instr_if));

    // ------------------------------------------------------------ IF/ID reg
    wire [31:0] instr_id, pc_plus4_id;

    IF_ID if_id (
        .clk(clk), .reset(reset),
        .enable(if_id_write),
        .flush(flush_if_id),
        .instr_in(instr_if), .pc_plus4_in(pc_plus4_if),
        .instr_out(instr_id), .pc_plus4_out(pc_plus4_id)
    );

    // ------------------------------------------------------------------ ID
    wire [5:0]  opcode_id = instr_id[31:26];
    wire [4:0]  rs_id     = instr_id[25:21];
    wire [4:0]  rt_id     = instr_id[20:16];
    wire [4:0]  rd_id     = instr_id[15:11];
    wire [5:0]  funct_id  = instr_id[5:0];
    wire [31:0] imm_id    = {{16{instr_id[15]}}, instr_id[15:0]};
    wire        c_reg_dst, c_alu_src, c_mem_read, c_mem_write;
```

```
wire        c_mem_to_reg, c_reg_write, c_branch, c_branch_ne;
wire [2:0]  c_alu_control;

control_unit cu (
    .opcode(opcode_id), .funct(funct_id),
    .reg_dst(c_reg_dst), .alu_src(c_alu_src), .alu_control(c_alu_control),
    .mem_read(c_mem_read), .mem_write(c_mem_write),
    .mem_to_reg(c_mem_to_reg), .reg_write(c_reg_write),
    .branch(c_branch), .branch_ne(c_branch_ne)
);

wire [31:0] rf_rd1, rf_rd2;
register_file rf (
    .clk(clk), .reset(reset),
    .reg_write(wb_reg_write),
    .read_reg1(rs_id), .read_reg2(rt_id),
    .write_reg(wb_write_reg), .write_data(wb_data),
    .read_data1(rf_rd1), .read_data2(rf_rd2)
);

// WB -> ID bypass (producer in WB, consumer in ID in the same cycle)
wire byp1 = wb_reg_write && (wb_write_reg != 5'd0) && (wb_write_reg == rs_id);
wire byp2 = wb_reg_write && (wb_write_reg != 5'd0) && (wb_write_reg == rt_id);
wire [31:0] rd1_id = byp1 ? wb_data : rf_rd1;
wire [31:0] rd2_id = byp2 ? wb_data : rf_rd2;

// ------------------------------------------------------------ ID/EX reg
ID_EX id_ex (
    .clk(clk), .reset(reset),
    .enable(1'b1),
    .control_mux(control_mux_hdu),   // load-use bubble
    .flush(flush_id_ex),             // taken-branch flush
    .pc_plus4_in(pc_plus4_id),
    .read_data1_in(rd1_id), .read_data2_in(rd2_id),
    .sign_ext_imm_in(imm_id),
    .rs_in(rs_id), .rt_in(rt_id), .rd_in(rd_id),
    .reg_dst_in(c_reg_dst), .alu_src_in(c_alu_src),
    .alu_control_in(c_alu_control),
    .mem_read_in(c_mem_read), .mem_write_in(c_mem_write),
    .mem_to_reg_in(c_mem_to_reg), .reg_write_in(c_reg_write),
    .branch_in(c_branch),
    .pc_plus4_out(ex_pc4),
    .read_data1_out(ex_rd1), .read_data2_out(ex_rd2),
    .sign_ext_imm_out(ex_imm),
    .rs_out(ex_rs), .rt_out(ex_rt), .rd_out(ex_rd),
    .reg_dst_out(ex_reg_dst), .alu_src_out(ex_alu_src),
    .alu_control_out(ex_alu_control),
    .mem_read_out(ex_mem_read), .mem_write_out(ex_mem_write),
    .mem_to_reg_out(ex_mem_to_reg), .reg_write_out(ex_reg_write),
    .branch_out(ex_branch)
);

// ID_EX has a single branch bit, so BEQ-vs-BNE travels in this 1-bit
// register. It only matters when ex_branch is 1; bubbles/flushes clear
// ex_branch, so no extra flush/stall handling is needed.
reg ex_is_bne;
always @(posedge clk or posedge reset) begin
    if (reset) ex_is_bne <= 1'b0;
    else       ex_is_bne <= c_branch_ne;
end

// Hazard detection (needs ID/EX outputs, so it sits after ID_EX)
hazard_detection_unit hdu (
    .id_ex_memread(ex_mem_read),
    .id_ex_rt(ex_rt),
    .if_id_opcode(opcode_id),
    .if_id_rs(rs_id),
    .if_id_rt(rt_id),
    .branch_taken(branch_taken_ex),
    .pc_write(pc_write_hdu),
    .if_id_write(if_id_write),
    .control_mux(control_mux_hdu),
    .flush_if_id(flush_if_id),
    .flush_id_ex(flush_id_ex)
);

// ------------------------------------------------------------------ EX
wire [1:0] forward_a, forward_b;

forwarding_unit fwd (
    .id_ex_rs(ex_rs), .id_ex_rt(ex_rt),
    .ex_mem_rd(mem_write_reg), .ex_mem_regwrite(mem_reg_write),
    .mem_wb_rd(wb_write_reg),  .mem_wb_regwrite(wb_reg_write),
    .forward_a(forward_a), .forward_b(forward_b)
);
```

```
    wire [31:0] fwd_a_val = (forward_a == 2'b10) ? mem_alu_result :
                            (forward_a == 2'b01) ? wb_data        : ex_rd1;
    wire [31:0] fwd_b_val = (forward_b == 2'b10) ? mem_alu_result :
                            (forward_b == 2'b01) ? wb_data        : ex_rd2;
    wire [31:0] alu_b     = ex_alu_src ? ex_imm : fwd_b_val;

    wire [31:0] alu_result_ex;
    wire        zero_ex;
    alu alu_u (
        .a(fwd_a_val), .b(alu_b), .alu_control(ex_alu_control),
        .result(alu_result_ex), .zero(zero_ex)
    );

    // Branch resolved here: BEQ taken on zero, BNE taken on not-zero
    assign branch_taken_ex  = ex_branch & (ex_is_bne ? ~zero_ex : zero_ex);
    assign branch_target_ex = ex_pc4 + (ex_imm << 2);

    wire [4:0] write_reg_ex = ex_reg_dst ? ex_rd : ex_rt;

    // ------------------------------------------------------------ EX/MEM reg
    // Branch fields are carried for completeness but unused: branches are
    // already resolved in EX.
    wire [31:0] unused_mem_branch_target;
    wire        unused_mem_zero, unused_mem_branch;

    EX_MEM ex_mem (
        .clk(clk), .reset(reset),
        .enable(1'b1),
        .flush(1'b0),
        .branch_target_in(branch_target_ex), .zero_in(zero_ex),
        .alu_result_in(alu_result_ex), .write_data_in(fwd_b_val),
        .write_reg_in(write_reg_ex),
        .mem_read_in(ex_mem_read), .mem_write_in(ex_mem_write),
        .mem_to_reg_in(ex_mem_to_reg), .reg_write_in(ex_reg_write),
        .branch_in(ex_branch),
        .branch_target_out(unused_mem_branch_target), .zero_out(unused_mem_zero),
        .alu_result_out(mem_alu_result), .write_data_out(mem_write_data),
        .write_reg_out(mem_write_reg),
        .mem_read_out(mem_mem_read), .mem_write_out(mem_mem_write),
        .mem_to_reg_out(mem_mem_to_reg), .reg_write_out(mem_reg_write),
        .branch_out(unused_mem_branch)
    );

    // ------------------------------------------------------------------ MEM
    data_memory dmem (
        .clk(clk),
        .mem_read(mem_mem_read), .mem_write(mem_mem_write),
        .address(mem_alu_result), .write_data(mem_write_data),
        .read_data(mem_read_data)
    );

    // ------------------------------------------------------------ MEM/WB reg
    MEM_WB mem_wb (
        .clk(clk), .reset(reset),
        .enable(1'b1), .flush(1'b0),
        .read_data_in(mem_read_data), .alu_result_in(mem_alu_result),
        .write_reg_in(mem_write_reg),
        .mem_to_reg_in(mem_mem_to_reg), .reg_write_in(mem_reg_write),
        .read_data_out(wb_read_data), .alu_result_out(wb_alu_result),
        .write_reg_out(wb_write_reg),
        .mem_to_reg_out(wb_mem_to_reg), .reg_write_out(wb_reg_write)
    );

    // ------------------------------------------------------------------- WB
    assign wb_data = wb_mem_to_reg ? wb_read_data : wb_alu_result;

    // ---------------------------------------------------------------
    // Hazard indicator wires (for waveforms and testbench counters)
    // ---------------------------------------------------------------
    // DATA: an instruction that really executes is using a forwarded operand
    wire ex_valid = ex_reg_write | ex_mem_write | ex_branch;
    wire data_hazard_forward = ex_valid & ((forward_a != 2'b00) | (forward_b != 2'b00));
    // DATA: load-use stall in progress
    wire data_hazard_loaduse = ~pc_write_hdu;
    // CONTROL: taken branch being resolved (2 wrong-path instructions flushed)
    wire control_hazard      = branch_taken_ex;
    // STRUCTURAL: would conflict if memory were unified (fetch + load/store same cycle)
    wire struct_hazard_mem   = mem_mem_read | mem_mem_write;
    // STRUCTURAL: register file written and read (same reg) in one cycle
    wire struct_hazard_rf    = wb_reg_write && (wb_write_reg != 5'd0) &&
                               ((wb_write_reg == rs_id) || (wb_write_reg == rt_id));
    wire struct_hazard       = struct_hazard_mem | struct_hazard_rf;

endmodule
```

**control_unit.v** - Main control unit (instruction decode)

```
// Main control unit. Decodes opcode/funct in the ID stage.
// alu_control uses the encoding of the repo's alu.v:
//   000 add, 001 sub, 010 and, 011 or, 100 slt
module control_unit (
    input  wire [5:0] opcode,
    input  wire [5:0] funct,
    output reg        reg_dst,
    output reg        alu_src,
    output reg [2:0]  alu_control,
    output reg        mem_read,
    output reg        mem_write,
    output reg        mem_to_reg,
    output reg        reg_write,
    output reg        branch,      // BEQ or BNE
    output reg        branch_ne    // 1 = BNE (taken when operands differ)
);
    localparam OP_RTYPE = 6'b000000;
    localparam OP_LW    = 6'b100011;
    localparam OP_SW    = 6'b101011;
    localparam OP_BEQ   = 6'b000100;
    localparam OP_BNE   = 6'b000101;
    localparam OP_ADDI  = 6'b001000;

    always @(*) begin
        reg_dst     = 1'b0;
        alu_src     = 1'b0;
        alu_control = 3'b000;
        mem_read    = 1'b0;
        mem_write   = 1'b0;
        mem_to_reg  = 1'b0;
        reg_write   = 1'b0;
        branch      = 1'b0;
        branch_ne   = 1'b0;

        case (opcode)
            OP_RTYPE: begin
                reg_dst   = 1'b1;
                reg_write = 1'b1;
                case (funct)
                    6'h20:   alu_control = 3'b000; // add
                    6'h22:   alu_control = 3'b001; // sub
                    6'h24:   alu_control = 3'b010; // and
                    6'h25:   alu_control = 3'b011; // or
                    6'h2A:   alu_control = 3'b100; // slt
                    default: reg_write   = 1'b0;   // nop (sll $0,$0,0) / unsupported
                endcase
            end
            OP_LW: begin
                alu_src    = 1'b1;
                mem_read   = 1'b1;
                mem_to_reg = 1'b1;
                reg_write  = 1'b1;
            end
            OP_SW: begin
                alu_src   = 1'b1;
                mem_write = 1'b1;
            end
            OP_BEQ: begin
                alu_control = 3'b001; // subtract; zero flag decides
                branch      = 1'b1;
            end
            OP_BNE: begin
                alu_control = 3'b001; // subtract; NOT zero decides
                branch      = 1'b1;
                branch_ne   = 1'b1;
            end
            OP_ADDI: begin
                alu_src   = 1'b1;
                reg_write = 1'b1;
            end
            default: ;
        endcase
    end
endmodule
```

**hazard_detection_unit.v** - Hazard detection unit: load-use stall and branch flush

```
module hazard_detection_unit (
    // Instruction currently in the ID/EX pipeline register
    input wire       id_ex_memread,
    input wire [4:0] id_ex_rt,

    // Instruction currently in the IF/ID pipeline register
    input wire [5:0] if_id_opcode,
    input wire [4:0] if_id_rs,
    input wire [4:0] if_id_rt,

    // Branch decision
    input wire branch_taken,

    // Pipeline control outputs
    output reg pc_write,
    output reg if_id_write,
    output reg control_mux,

    // Control hazard outputs
    output reg flush_if_id,
    output reg flush_id_ex
);

    // MIPS opcodes
    localparam OP_RTYPE = 6'b000000;
    localparam OP_LW    = 6'b100011;
    localparam OP_SW    = 6'b101011;
    localparam OP_BEQ   = 6'b000100;
    localparam OP_BNE   = 6'b000101;
    localparam OP_ADDI  = 6'b001000;

    reg uses_rs;
    reg uses_rt;

    always @(*) begin
        // Determine which source registers the
        // instruction in IF/ID actually uses.
        uses_rs = 1'b0;
        uses_rt = 1'b0;

        case (if_id_opcode)
            OP_RTYPE: begin
                uses_rs = 1'b1;
                uses_rt = 1'b1;
            end

            OP_LW, OP_ADDI: begin
                uses_rs = 1'b1;
            end

            OP_SW, OP_BEQ, OP_BNE: begin
                uses_rs = 1'b1;
                uses_rt = 1'b1;
            end

            default: begin
                uses_rs = 1'b0;
                uses_rt = 1'b0;
            end
        endcase

        // Default: allow normal pipeline operation.
        pc_write     = 1'b1;
        if_id_write  = 1'b1;
        control_mux  = 1'b0;

        flush_if_id  = 1'b0;
        flush_id_ex  = 1'b0;

        // Detect a control hazard.
        if (branch_taken) begin
            flush_if_id = 1'b1;
            flush_id_ex = 1'b1;
        end

        // Detect a load-use hazard.
        else if (id_ex_memread &&
                 (id_ex_rt != 5'd0) &&
                 ((uses_rs && (id_ex_rt == if_id_rs)) ||
                  (uses_rt && (id_ex_rt == if_id_rt)))) begin

            pc_write     = 1'b0;
```

```
            if_id_write  = 1'b0;
            control_mux  = 1'b1;
        end
    end

endmodule
```

## **forwarding_unit.v** - Forwarding unit

```
module forwarding_unit (
    // Source registers of the instruction in EX
    input  wire [4:0] id_ex_rs,
    input  wire [4:0] id_ex_rt,

    // Destination register of the instruction in MEM
    input  wire [4:0] ex_mem_rd,
    input  wire       ex_mem_regwrite,

    // Destination register of the instruction in WB
    input  wire [4:0] mem_wb_rd,
    input  wire       mem_wb_regwrite,

    // Forwarding controls for the ALU inputs
    output reg  [1:0] forward_a,
    output reg  [1:0] forward_b
);

    always @(*) begin
        // Default: use original register operands
        forward_a = 2'b00;
        forward_b = 2'b00;

        // Forward operand A from the MEM stage
        if (ex_mem_regwrite &&
            (ex_mem_rd != 5'd0) &&
            (ex_mem_rd == id_ex_rs)) begin
            forward_a = 2'b10;
        end

        // Forward operand B from the MEM stage
        if (ex_mem_regwrite &&
            (ex_mem_rd != 5'd0) &&
            (ex_mem_rd == id_ex_rt)) begin
            forward_b = 2'b10;
        end

        // Forward operand A from the WB stage
        // only if the MEM stage did not already match
        if (mem_wb_regwrite &&
            (mem_wb_rd != 5'd0) &&
            (mem_wb_rd == id_ex_rs) &&
            !(ex_mem_regwrite &&
              (ex_mem_rd != 5'd0) &&
              (ex_mem_rd == id_ex_rs))) begin
            forward_a = 2'b01;
        end

        // Forward operand B from the WB stage
        // only if the MEM stage did not already match
        if (mem_wb_regwrite &&
            (mem_wb_rd != 5'd0) &&
            (mem_wb_rd == id_ex_rt) &&
            !(ex_mem_regwrite &&
              (ex_mem_rd != 5'd0) &&
              (ex_mem_rd == id_ex_rt))) begin
            forward_b = 2'b01;
        end
    end

endmodule
```

## **alu.v** - ALU with zero flag

```
module alu (
    input  [31:0] a,
    input  [31:0] b,
    input  [2:0] alu_control,
    output reg [31:0] result,
    output zero
);

    always @(*) begin
```

```
        case (alu_control)

            3'b000: result = a + b;                   // ADD
            3'b001: result = a - b;                   // SUB
            3'b010: result = a & b;                   // AND
            3'b011: result = a | b;                   // OR
            3'b100: result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0; // SLT

            default: result = 32'b0;

        endcase
    end

    // Zero flag: used for branch comparison
    assign zero = (result == 32'b0);

endmodule
```

## **program_counter.v** - Program counter with branch-target load

```
module program_counter (
    input clk,
    input reset,
    input pc_write,

    input branch_taken,
    input [31:0] branch_target,

    input [31:0] next_pc,
    output reg [31:0] pc
);

    always @(posedge clk or posedge reset) begin

        if (reset) begin
            pc <= 32'b0;
        end

        // Branch has priority over normal PC update or stall
        else if (branch_taken) begin
            pc <= branch_target;
        end

        // Normal sequential PC update
        else if (pc_write) begin
            pc <= next_pc;
        end

    end

endmodule
```

# TEST BENCH FILE:

The testbench (tb_mips_pipeline.v) loads the test program below into the instruction memory, resets and clocks the processor, logs every hazard it detects, and then checks the final register and memory contents and the number of stalls and taken branches. The program is written so that every hazard type occurs at least once.

## Test program (program.hex)

| # | Machine code | Instruction | Hazard exercised / purpose |
|---|---|---|---|
| 0 | 20010005 | addi $1,$0,5 | initialise $1 |
| 1 | 00211020 | add $2,$1,$1 | RAW: forwarded from EX/MEM |
| 2 | 00411820 | add $3,$2,$1 | RAW: forwarded from EX/MEM and MEM/WB |
| 3 | AC030000 | sw $3,0($0) | store of a forwarded value |
| 4 | 8C040000 | lw $4,0($0) | load |
| 5 | 00812820 | add $5,$4,$1 | LOAD-USE hazard: 1-cycle stall |
| 6 | 20080001 | addi $8,$0,1 | initialise $8 |
| 7 | 21080001 | addi $8,$8,1 | RAW chain |
| 8 | 21080001 | addi $8,$8,1 | RAW chain |
| 9 | 01084820 | add $9,$8,$8 | RAW |
| 10 | 01215022 | sub $10,$9,$1 | RAW |
| 11 | 00000000 | nop | nop |
| 12 | 00000000 | nop | nop |
| 13 | 014A5820 | add $11,$10,$10 | RAW 3 instructions apart: WB->ID bypass |
| 14 | 10210001 | beq $1,$1,+1 | BEQ taken (skips 1) |
| 15 | 20060063 | addi $6,$0,99 | flushed: must not write $6 |
| 16 | 20070007 | addi $7,$0,7 | branch target |
| 17 | AC050004 | sw $5,4($0) | store |
| 18 | 200C0007 | addi $12,$0,7 | initialise $12 |
| 19 | 11870001 | beq $12,$7,+1 | BEQ taken, operand forwarded |
| 20 | 200D0063 | addi $13,$0,99 | flushed: must not write $13 |
| 21 | 200E0001 | addi $14,$0,1 | initialise $14 |
| 22 | 8C120000 | lw $18,0($0) | load |
| 23 | 12430001 | beq $18,$3,+1 | LOAD-USE into a branch: stall, then taken |
| 24 | 20130063 | addi $19,$0,99 | flushed: must not write $19 |
| 25 | 20140001 | addi $20,$0,1 | branch target |
| 26 | 15C00002 | bne $14,$0,+2 | BNE taken (skips 2) |
| 27 | 200F0063 | addi $15,$0,99 | flushed by ID/EX flush ($15) |
| 28 | 20150063 | addi $21,$0,99 | flushed by IF/ID flush ($21) |
| 29 | 15CE0001 | bne $14,$14,+1 | BNE not taken |
| 30 | 20100009 | addi $16,$0,9 | executes (fall-through) |
| 31 | 20110002 | addi $17,$0,2 | executes |
| 32 | 1000FFFF | beq $0,$0,-1 | parks the CPU (branch to itself) |

### tb_mips_pipeline.v

```
`timescale 1ns/1ps
// Self-checking system testbench for mips_pipeline.
// Detects, logs and counts all three hazard classes, then checks results.
//
// Program under test (program.hex):
//  idx  instruction           exercises
//   0   addi $1,$0,5
//   1   add  $2,$1,$1         DATA  forward EX/MEM        ($2=10)
//   2   add  $3,$2,$1         DATA  forward MEM + WB      ($3=15)
//   3   sw   $3,0($0)         DATA  forwarded store data  (dmem[0]=15)
//   4   lw   $4,0($0)
```

```
//   5   add  $5,$4,$1          DATA  LOAD-USE stall #1     ($5=20)
//  6-8  addi $8 chain          DATA  back-to-back RAW      ($8=3)
//   9   add  $9,$8,$8                                      ($9=6)
//  10   sub  $10,$9,$1                                     ($10=1)
// 11-12 nop
//  13   add  $11,$10,$10       DATA  distance-3 (WB->ID bypass) ($11=2)
//  14   beq  $1,$1,+1          CONTROL taken #1
//  15   addi $6,$0,99          flushed                     ($6=0)
//  16   addi $7,$0,7
//  17   sw   $5,4($0)                                      (dmem[1]=20)
//  18   addi $12,$0,7
//  19   beq  $12,$7,+1         CONTROL taken #2, forwarded operands
//  20   addi $13,$0,99         flushed                     ($13=0)
//  21   addi $14,$0,1
//  22   lw   $18,0($0)
//  23   beq  $18,$3,+1         DATA load-use stall #2 into a branch, then
//                              CONTROL taken #3            ($18=15)
//  24   addi $19,$0,99         flushed                     ($19=0)
//  25   addi $20,$0,1                                      ($20=1)
//  26   bne  $14,$0,+2         CONTROL taken #4 (BNE, skips TWO instrs)
//  27   addi $15,$0,99         flushed by ID/EX flush      ($15=0)
//  28   addi $21,$0,99         flushed by IF/ID flush      ($21=0)
//  29   bne  $14,$14,+1        BNE NOT taken
//  30   addi $16,$0,9          executes                    ($16=9)
//  31   addi $17,$0,2                                      ($17=2)
//  32   beq  $0,$0,-1          parks the CPU in a loop

module tb_mips_pipeline;
    reg clk   = 1'b0;
    reg reset = 1'b1;

    integer errors  = 0;
    integer cycle   = 0;
    integer n_fwd    = 0;   // cycles using a forwarded operand
    integer n_stall  = 0;   // load-use stall cycles
    integer n_branch = 0;   // taken branches (excluding the parking loop)
    integer n_struct = 0;   // structural-hazard conditions detected (avoided)

    mips_pipeline dut (.clk(clk), .reset(reset));

    always #5 clk = ~clk;                       // 100 MHz

    // Parking loop (idx 32): its PC+4 is 132. Ignore it when counting.
    wire in_park_loop = (dut.ex_pc4 == 32'd132);

    always @(posedge clk) if (!reset) cycle <= cycle + 1;

    // ---------------- Hazard detection log + counters ----------------
    // Sampled at the falling edge (mid-cycle, signals are stable). The time printed is the
    // START of the clock cycle in which the hazard is active, so it matches the waveform.
    always @(negedge clk) if (!reset) begin
        if (dut.data_hazard_loaduse) begin
            n_stall <= n_stall + 1;
            $display("[%0d ns] DATA HAZARD    load-use: stall 1 cycle (PC and IF/ID held, bubble to EX)", $time - 5);
        end
        if (dut.data_hazard_forward) begin
            n_fwd <= n_fwd + 1;
            $display("[%0d ns] DATA HAZARD    forwarding  A=%0d B=%0d  (2=from EX/MEM, 1=from MEM/WB)",
                     $time - 5, dut.forward_a, dut.forward_b);
        end
        if (dut.control_hazard && !in_park_loop) begin
            n_branch <= n_branch + 1;
            $display("[%0d ns] CONTROL HAZARD branch taken -> target %h, flush IF/ID + ID/EX",
                     $time - 5, dut.branch_target_ex);
        end
        if (dut.struct_hazard) begin
            n_struct <= n_struct + 1;
`ifdef VERBOSE_STRUCT
            $display("[%0d ns] STRUCTURAL     mem=%b regfile=%b (avoided: separate I/D memory, bypass)",
                     $time - 5, dut.struct_hazard_mem, dut.struct_hazard_rf);
`endif
        end
    end

    // Load program (#1 so it lands after instruction_memory's own init block)
    initial begin
        #1 $readmemh("program.hex", dut.imem.memory, 0, 32);
    end

    task check_reg(input integer n, input [31:0] expected);
        begin
            if (dut.rf.registers[n] !== expected) begin
                $display("FAIL: $%0d = %h, expected %h", n, dut.rf.registers[n], expected);
                errors = errors + 1;
            end else
```

```
                $display("PASS: $%0d = %0d", n, dut.rf.registers[n]);
            end
        endtask

        task check_mem(input integer idx, input [31:0] expected);
            begin
                if (dut.dmem.memory[idx] !== expected) begin
                    $display("FAIL: dmem[%0d] = %h, expected %h", idx, dut.dmem.memory[idx], expected);
                    errors = errors + 1;
                end else
                    $display("PASS: dmem[%0d] = %0d", idx, dut.dmem.memory[idx]);
            end
        endtask

        task check_count(input [255:0] name, input integer got, input integer expected);
            begin
                if (got !== expected) begin
                    $display("FAIL: %0s = %0d, expected %0d", name, got, expected);
                    errors = errors + 1;
                end else
                    $display("PASS: %0s = %0d", name, got);
            end
        endtask

        initial begin
            $dumpfile("waves.vcd");
            $dumpvars(0, tb_mips_pipeline);

            #22 reset = 1'b0;
            #800;                                   // ~80 cycles; program needs ~45

            $display("---- Results after %0d cycles ----", cycle);
            check_reg(1,  32'd5);
            check_reg(2,  32'd10);
            check_reg(3,  32'd15);
            check_reg(4,  32'd15);
            check_reg(5,  32'd20);
            check_reg(6,  32'd0);     // flushed
            check_reg(7,  32'd7);
            check_reg(8,  32'd3);
            check_reg(9,  32'd6);
            check_reg(10, 32'd1);
            check_reg(11, 32'd2);
            check_reg(12, 32'd7);
            check_reg(13, 32'd0);     // flushed
            check_reg(14, 32'd1);
            check_reg(15, 32'd0);     // flushed (BNE taken)
            check_reg(16, 32'd9);     // BNE not taken -> executed
            check_reg(17, 32'd2);
            check_reg(18, 32'd15);
            check_reg(19, 32'd0);     // flushed
            check_reg(20, 32'd1);
            check_reg(21, 32'd0);     // flushed by IF/ID flush (2nd skipped instr)
            check_reg(0,  32'd0);     // $0 never written

            check_mem(0, 32'd15);
            check_mem(1, 32'd20);

            $display("---- Hazard summary ----");
            check_count("load-use stalls ", n_stall,  2);
            check_count("taken branches  ", n_branch, 4);
            if (n_fwd > 0) $display("PASS: forwarding used in %0d cycles", n_fwd);
            else begin $display("FAIL: forwarding never used"); errors = errors + 1; end
            if (n_struct > 0)
                $display("PASS: structural conflicts detected %0d times and avoided (separate I/D memory + bypass)", n_struct);
            else begin $display("FAIL: no structural conflict condition seen"); errors = errors + 1; end

            if (errors == 0) $display("*** ALL TESTS PASSED ***");
            else             $display("*** %0d TEST(S) FAILED ***", errors);

    `ifdef BATCH
            $finish;
    `else
            $stop;
    `endif
        end
    endmodule
```

# SCREEN SHOT OF THE OUTPUT:

*Figure 1: QuestaSim transcript of the testbench. Each hazard is logged with the time at which it is detected. All 28 checks pass: 22 registers, 2 data-memory words and 4 hazard-summary checks (2 load-use stalls, 4 taken branches, forwarding used in 10 cycles, structural conflicts detected and avoided).*

```
# [45 ns] DATA HAZARD    forwarding  A=2 B=2  (2=from EX/MEM, 1=from MEM/WB)
# [55 ns] DATA HAZARD    forwarding  A=2 B=1  (2=from EX/MEM, 1=from MEM/WB)
# [65 ns] DATA HAZARD    forwarding  A=0 B=2  (2=from EX/MEM, 1=from MEM/WB)
# [75 ns] DATA HAZARD    load-use: stall 1 cycle (PC and IF/ID held, bubble to EX)
# [95 ns] DATA HAZARD    forwarding  A=1 B=0  (2=from EX/MEM, 1=from MEM/WB)
# [115 ns] DATA HAZARD    forwarding  A=2 B=2  (2=from EX/MEM, 1=from MEM/WB)
# [125 ns] DATA HAZARD    forwarding  A=2 B=2  (2=from EX/MEM, 1=from MEM/WB)
# [135 ns] DATA HAZARD    forwarding  A=2 B=2  (2=from EX/MEM, 1=from MEM/WB)
# [145 ns] DATA HAZARD    forwarding  A=2 B=0  (2=from EX/MEM, 1=from MEM/WB)
# [185 ns] CONTROL HAZARD branch taken -> target 00000040, flush IF/ID + ID/EX
# [245 ns] DATA HAZARD    forwarding  A=2 B=0  (2=from EX/MEM, 1=from MEM/WB)
# [245 ns] CONTROL HAZARD branch taken -> target 00000054, flush IF/ID + ID/EX
# [285 ns] DATA HAZARD    load-use: stall 1 cycle (PC and IF/ID held, bubble to EX)
# [305 ns] DATA HAZARD    forwarding  A=1 B=0  (2=from EX/MEM, 1=from MEM/WB)
# [305 ns] CONTROL HAZARD branch taken -> target 00000064, flush IF/ID + ID/EX
# [345 ns] CONTROL HAZARD branch taken -> target 00000074, flush IF/ID + ID/EX
# ---- Results after 80 cycles ----
# PASS: $1 = 5
# PASS: $2 = 10
# PASS: $3 = 15
# PASS: $4 = 15
# PASS: $5 = 20
# PASS: $6 = 0
# PASS: $7 = 7
# PASS: $8 = 3
# PASS: $9 = 6
# PASS: $10 = 1
# PASS: $11 = 2
# PASS: $12 = 7
# PASS: $13 = 0
# PASS: $14 = 1
# PASS: $15 = 0
# PASS: $16 = 9
# PASS: $17 = 2
# PASS: $18 = 15
# PASS: $19 = 0
# PASS: $20 = 1
# PASS: $21 = 0
# PASS: $0 = 0
# PASS: dmem[0] = 15
# PASS: dmem[1] = 20
# ---- Hazard summary ----
# PASS: load-use stalls  = 2
# PASS: taken branches   = 4
# PASS: forwarding used in 10 cycles
# PASS: structural conflicts detected 6 times and avoided (separate I/D memory + bypass)
# *** ALL TESTS PASSED ***
```

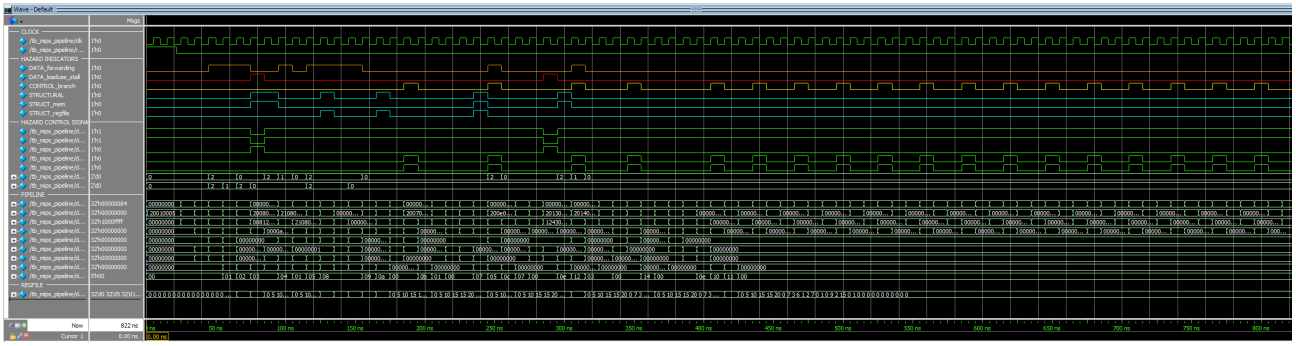


*Figure 2: QuestaSim waveform of the hazard indicators for the whole run. Red: the two load-use stalls (75 ns and 285 ns). Yellow: the four taken branches (the later repeated pulses are the final parking loop). Orange: forwarding in use. Cyan: cycles in which a structural conflict would have occurred and was avoided.*

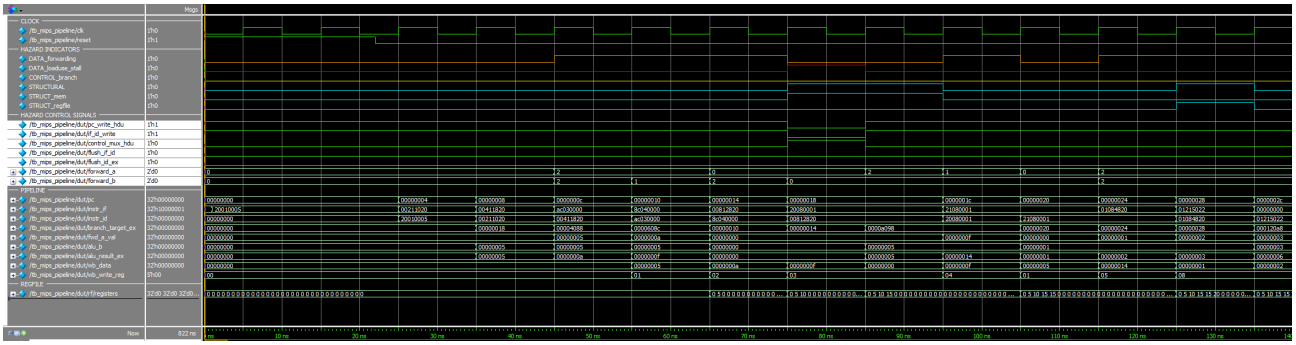


*Figure 3: Load-use hazard (data). lw $4,0($0) is followed immediately by add $5,$4,$1. For one cycle (75-85 ns) pc_write and if_id_write drop to 0, holding the PC and the IF/ID register, and control_mux inserts a bubble into EX. The stalled add then reaches EX one cycle later and receives the loaded value from MEM/WB (forward_a = 1).*

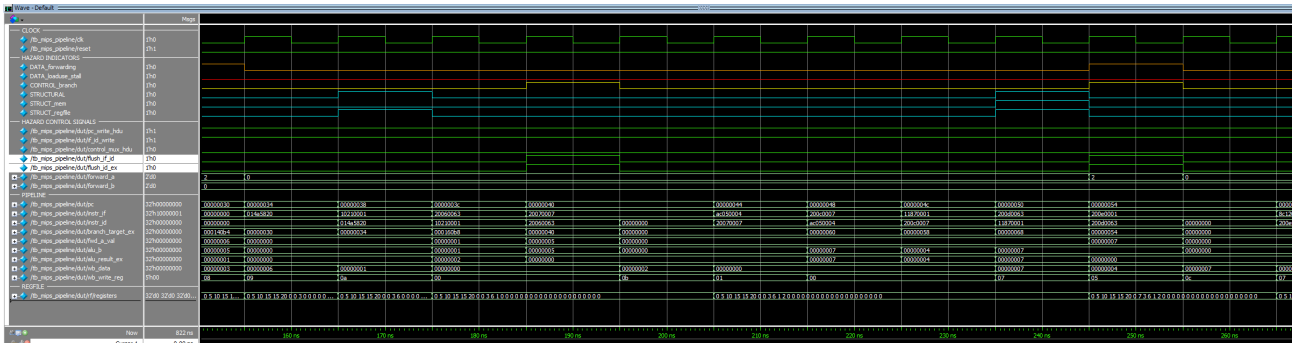


*Figure 4: Control hazard. beq $1,$1,+1 is resolved in EX at 185 ns: control_hazard goes high, flush_if_id and flush_id_ex clear the wrong-path instructions (the addi $6,$0,99 in ID becomes a nop) and the PC is loaded with the branch target 0x40.*

# REFERENCES


[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface (MIPS Edition)*, Morgan Kaufmann. (Chapter on the processor and pipelining: forwarding, load-use stalls and branch hazards.)

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann. (Appendix C: Pipelining.)

[3] M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, Pearson.

[4] S. Harris and D. Harris, *Digital Design and Computer Architecture*, Morgan Kaufmann. (Pipelined MIPS processor with hazard units in Verilog.)

[5] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, Prentice Hall.

[6] J. E. Smith, "A Study of Branch Prediction Strategies," *Proc. 8th Annual Symposium on Computer Architecture*, 1981.

[7] Siemens EDA, *QuestaSim User's Manual*; Intel, *Questa-Intel FPGA Edition documentation*.